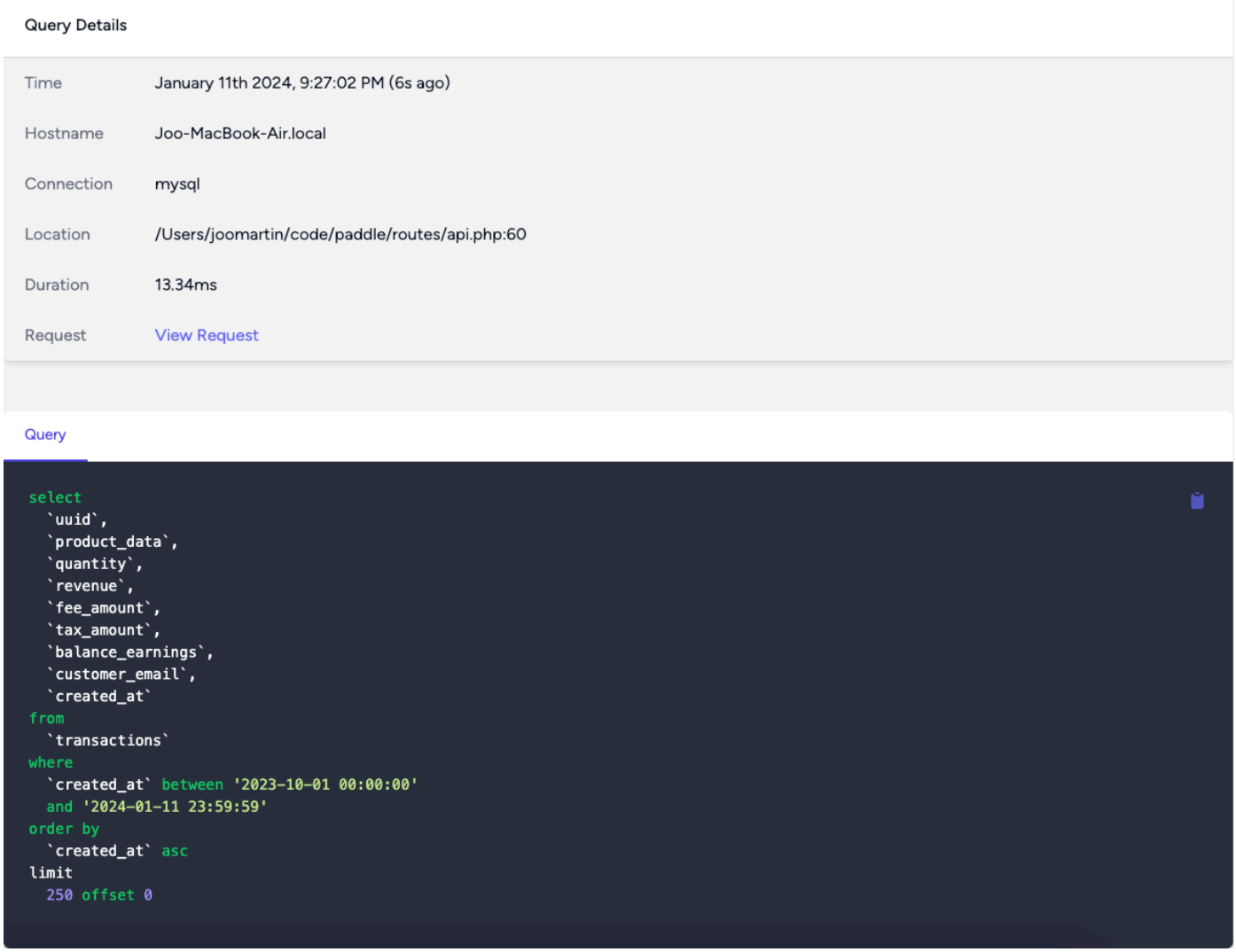


Let's dig deeper by running `explain` on the original query:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	transactions	NULL	range	user_id,created_at	created_at	5	NULL	93909	9.88	Using index condition; Using where

- Type `range` means that MySQL was able to limit the number of rows that it has to go through. So it doesn't need to scan the full table. It's a good thing.
- `possible_keys` means MySQL considered using either the `user_id` or the `created_at` keys.
- `key` means that it decided to use the `created_at` key
- `rows` means it went through `93 909`.

Scanning only 94k rows is a good thing if we consider that there are 200k rows in the table but it's a bad thing if we know that the given user has only 10k transactions. All of this is because MySQL used the `created_at` index. And it used this index because we have the `order by created_at asc`. In the `created_at` index (which is a B-TREE but you can imagine it as another table) there are 200k rows sorted by the `created_at` date, something like that: